

Relations and finite state machines

Applied Algebra
Instructor: Tuan Tran

Why do we need relations?

So far, functions assign *one output* to each input.

Many situations are more flexible:

- a student can have several friends;
- a number can divide many numbers;
- two cities may be connected by a road.

We need a more general notion: **relations**.

Examples of relations

Let $X = \mathbb{Z}$.

$$x \sim y \quad \text{if } x = y,$$

$$x \sim y \quad \text{if } x \leq y,$$

$$x \sim y \quad \text{if } x \text{ divides } y,$$

$$x \sim y \quad \text{if } x \equiv y \pmod{3}.$$

Each describes a relationship between elements.

Each relation consists of certain pairs (x, y) .

For example,

$$x \leq y \iff (x, y) \text{ satisfies a condition.}$$

Conclusion. A relation can be viewed as a collection of pairs.

Definition of a relation

Let X and Y be sets. Their **Cartesian product** is

$$X \times Y = \{(x, y) : x \in X, y \in Y\}.$$

Example. If $X = \{1, 2\}$ and $Y = \{x, y\}$, then

$$X \times Y = \{(1, x), (1, y), (2, x), (2, y)\}.$$

A **relation** R from X to Y is a subset

$$R \subseteq X \times Y.$$

We write

$$xRy \quad \text{if } (x, y) \in R.$$

Relations on a set

If $X = Y$, then a relation is a subset

$$R \subseteq X \times X.$$

We call this a **relation on X** .

Examples.

- $\leq$ on $\mathbb{Z}$;
- equality on any set;
- congruence modulo n .

Example

Let $X = \{1, 2, 3, 4\}$.

Define

$$R = \{(1, 1), (1, 2), (2, 3), (3, 4), (4, 3)\}.$$

Then:

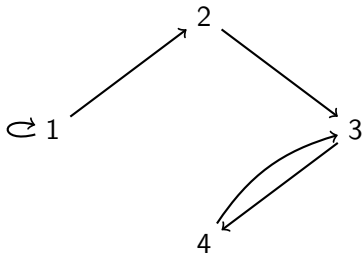
- $1R1, 1R2, 2R3, 3R4, 4R3$, but not $3R2$.

Relations as directed graphs

Let R be a relation on X . The **directed graph** of R has

- vertices X ,
- edges $x_1 \rightarrow x_2$ iff $(x_1, x_2) \in R$.

Example. Let $R = \{(1, 1), (1, 2), (2, 3), (3, 4), (4, 3)\}$.



Relations as adjacency matrices

Let R be a relation on a finite set $X = \{a_1, \dots, a_n\}$.

To specify R is equivalent to specifying its **adjacency matrix**

$$M_R = (m_{ij})_{1 \leq i, j \leq n}, \quad m_{ij} = \begin{cases} 1 & \text{if } (a_i, a_j) \in R, \\ 0 & \text{otherwise.} \end{cases}$$

Example. Let $X = \{1, 2, 3, 4\}$ and $R = \{(1, 1), (1, 2), (2, 3), (3, 4), (4, 3)\}$.

$$M_R = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Reflexive relations

A relation R on X is **reflexive** if

$$xRx \quad \text{for all } x \in X.$$

Examples.

- equality is reflexive;
- $\leq$ is reflexive.

Symmetric and antisymmetric relations

A relation R is

symmetric if

$$aRb \implies bRa,$$

antisymmetric if

$$aRb \text{ and } bRa \implies a = b.$$

Examples.

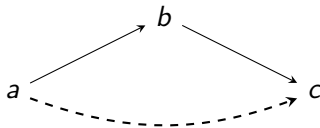
- equality is both symmetric and antisymmetric;
- “is a friend of” is symmetric but not antisymmetric;
- $a \leq b$ is antisymmetric but not symmetric.

Remark. R is symmetric if and only if its adjacency matrix M_R is symmetric.

Transitive relations

A relation R is **transitive** if

$$aRb \text{ and } bRc \implies aRc.$$



Examples.

- $\leq$ is transitive;
- divisibility is transitive.

Quick check

Which of the following relations are reflexive, symmetric, transitive?

- $a = b$ on $\mathbb{Z}$;
- $a \leq b$ on $\mathbb{Z}$;
- a divides b on $\mathbb{Z}$;
- $a \equiv b \pmod{3}$.

Equivalence relations

A relation E on a set X is an **equivalence relation** if it is:

- reflexive;
- symmetric;
- transitive.

Examples.

- equality;
- congruence modulo n .

Equivalence classes

Let E be an equivalence relation on X . For $x \in X$, the **equivalence class** of x is

$$[x]_E = \{y \in X : xEy\}.$$

Example. Consider congruence modulo 3 on $\mathbb{Z}$.

$$[0]_3 = \{\dots, -6, -3, 0, 3, 6, \dots\},$$

$$[1]_3 = \{\dots, -5, -2, 1, 4, 7, \dots\},$$

$$[2]_3 = \{\dots, -4, -1, 2, 5, 8, \dots\}.$$

Key properties of equivalence classes

Let R be an equivalence relation. Then

- $x \in [x]_E$;
- if xEy , then $[x]_E = [y]_E$;
- if $(x, y) \notin E$, then $[x]_E \cap [y]_E = \emptyset$.

Thus, distinct equivalence classes form a partition of X .

A **partition** of X is a collection of subsets such that:

- every element of X lies in one subset;
- the subsets are disjoint;
- none of them is empty.

Example. $X = \{1, 2, 3, 4, 5, 6\} = \{1, 2\} \cup \{3, 4\} \cup \{5, 6\}$.

Equivalence relations and partitions

equivalence relations on $X \iff$ partitions of X .

Theorem. Let X be a set.

(i) If $\{X_i : i \in I\}$ is a partition of X , then the relation E defined by

$$xEy \iff x, y \text{ lie in the same } X_i$$

is an equivalence relation.

(ii) Conversely, if E is an equivalence relation on X , then E induces a partition of X whose blocks are the equivalence classes

$$[x]_E = \{y \in X : yEx\}.$$

Finite State Machines

In 1937, **Alan Turing** introduced Turing machines, a model capable of performing any algorithmic computation.

We study a restricted class of such machines, called **finite state machines**.

Limitations.

- less computational power,
- no external memory.

Advantages.

- simple to construct,
- very important for applications.

What is a finite state machine?

A **finite state machine** consists of:

- a finite set of states,
- an input alphabet,
- a rule for moving between states.

At each step, the machine reads an input letter and updates its state.

Definition

A finite state machine is a triple $M = (S, A, \mu)$, where

- S is a finite set of states with initial state 0,
- A is an alphabet,
- $\mu : S \times A \rightarrow S$ is a transition function.

How a finite state machine works

Think of a machine reading an input word over A , one letter at a time.

- The machine starts in state 0.
- If it is in state s and reads $a \in A$,
- it moves to the state $\mu(s, a)$.
- It then proceeds to the next letter.

Thus, the current state records all information about the input read so far.

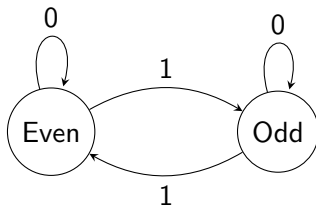
Example: parity machine

We read a binary string over $\{0, 1\}$.

Goal. Determine whether the number of 1s is even or odd.

States.

- Even,
- Odd.



How Does It Work?

Initialization. Start in the state **Even**.

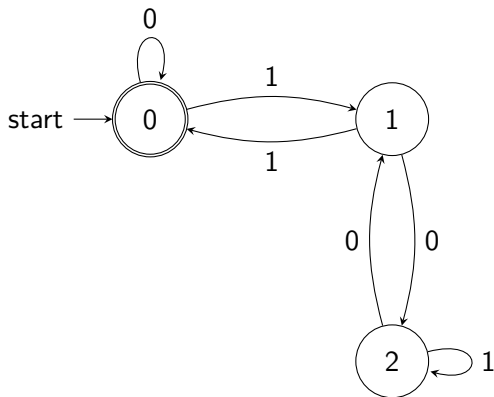
Update rule.

- Reading 0 leaves the state unchanged.
- Reading 1 switches between Even and Odd.

At any time, the current state records the parity of the number of 1s seen so far.

FSM for divisibility by 3

Goal. Recognize binary numbers divisible by 3.



Interpretation. State $r \in \{0, 1, 2\}$ records the remainder modulo 3 of the input read so far.

Why does it work?

Key observation. If the current remainder is r and we read a bit $b \in \{0, 1\}$, then the new remainder is

$$r \mapsto (2r + b) \bmod 3.$$

Explanation.

- Reading a new bit appends it to the right in binary,
- which multiplies the current number by 2 and adds b .

Conclusion. The machine correctly tracks the remainder modulo 3, and accepts exactly those inputs that end in state 0.

Design your own machine

Problem. Design a finite state machine that:

- reads binary strings;
- accepts strings with an even number of 0's.

Finite state automata

Finite state automata are a class of finite state machines that arise in many applications:

- control systems (e.g. cash machines, lifts),
- recognizing keywords in programming languages,
- pattern matching (e.g. passwords or PINs).

They are also fundamental in theoretical computer science and formal language theory.

Definition

A **finite state automaton** is a finite state machine $M = (S, A, \mu)$ together with a set $F \subseteq S$ of *accepting states*.

A sequence σ of letters in the alphabet A is **accepted** by M if, after reading σ , the machine ends in a state $s \in F$.

Example

Let $M = (S, A, \mu)$ be a finite state automaton with

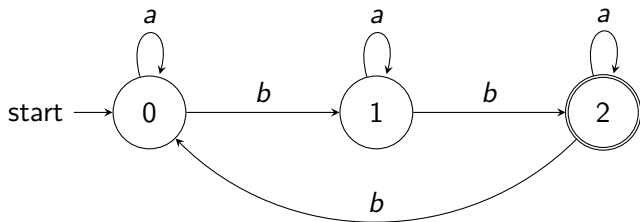
$$S = \{0, 1, 2\}, \quad A = \{a, b\}.$$

The transition function μ is given by:

	a	b
0	0	1
1	1	2
2	2	0

Let the set of accepting states be $F = \{2\}$.

State diagram and discussion



Observation.

- Reading a does not change the state.
- Reading b cycles through $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$.

Conclusion. The state records the number of b 's modulo 3. A word is accepted if and only if

$$\#b \equiv 2 \pmod{3}.$$